

NAUT ERP PLATFORM

Software Architecture & Project Scope Specification

Author: J. Pablo Velasquez

1. Executive Summary & Vision

The Naut platform is designed as a unified, web-based Enterprise Resource Planning (ERP) suite. Built with a Modular Monolith architecture, the application integrates specialized business management tools under a cohesive aerospace and terraforming-inspired interface. The initial launch focuses on establishing three core pillars that operate independently but share a centralized database to automate business operations seamlessly.

2. Core Modules (The Spokes)

A. Finance Analitica (The Ledger)

The financial heartbeat of the platform. Instead of relying solely on manual data entry, this module acts as a passive aggregator, listening to activities across the ecosystem to generate real-time financial insights.

- **Core Functions:** Tracking operating income, expenditure, and calculating net profit for Florida LLCs or business entities.
- **Visualization:** Rendered dashboards using Chart.js to map cash flow trends over time.
- **Automation:** Automatically updates balances when services are booked or inventory supplies are purchased.

B. Service Management (The Calendar)

The operational scheduling engine designed to manage logistical booking and provider assignments.

- **Core Functions:** Interactive calendars for blocking dates and managing recurring operations (e.g., mapping out property cleaning schedules or booking multi-day maintenance jobs).
- **Provider Logistics:** Assigning specific personnel to tasks and processing client reservations.
- **Cross-Talk Integration:** Upon completion or booking of a service, the module triggers a payment event that is automatically logged into the Finance module's ledger.

C. Product Manejo (Inventory Management)

The asset tracking system for managing physical goods, raw materials, and product sales.

- **Core Functions:** Monitoring stock levels in real-time, calculating the cost of goods sold, and managing supplier orders (e.g., tracking the depletion of bulk materials like pallets of clay or cleaning supplies).
- **Cross-Talk Integration:** Deducts from inventory counts dynamically when jobs in the Service module are executed. Purchasing new materials automatically logs an expense in the Finance module.

3. System Architecture

The platform relies on a **Node.js (Express)** backend with an **EJS** templating frontend, structuring logic using the Model-View-Controller (MVC) pattern. Data persistence is managed via a relational database (PostgreSQL/MySQL) to facilitate module cross-talk.

```
naut_platform/
├── core/           # The Shared Hub
│   ├── database/  # Relational DB Config
│   ├── routes/    # Global routing (Auth, Dashboard)
│   └── views/      # Shared UI templates
├── modules/       # The Application Spokes
│   ├── finance/   # Controllers, Models, Routes
│   ├── services/  # Controllers, Models, Routes
│   └── inventory/ # Controllers, Models, Routes
├── public/        # Static Assets (CSS, Logos)
└── server.js      # Application Entry Point
```

4. Phased Implementation Plan

- **Phase 1: Foundation & Data Modeling.** Establish the Node.js environment and design the relational database schema outlining exactly how a *Transaction*, *Service Booking*, and *Inventory Item* interact.
- **Phase 2: Core Platform Routing.** Build the core hub, establishing the unified dashboard view and basic navigation structure.
- **Phase 3: Module Development.** Construct the isolated logic for Finance, Services, and Inventory, ensuring the models accurately query the shared database.